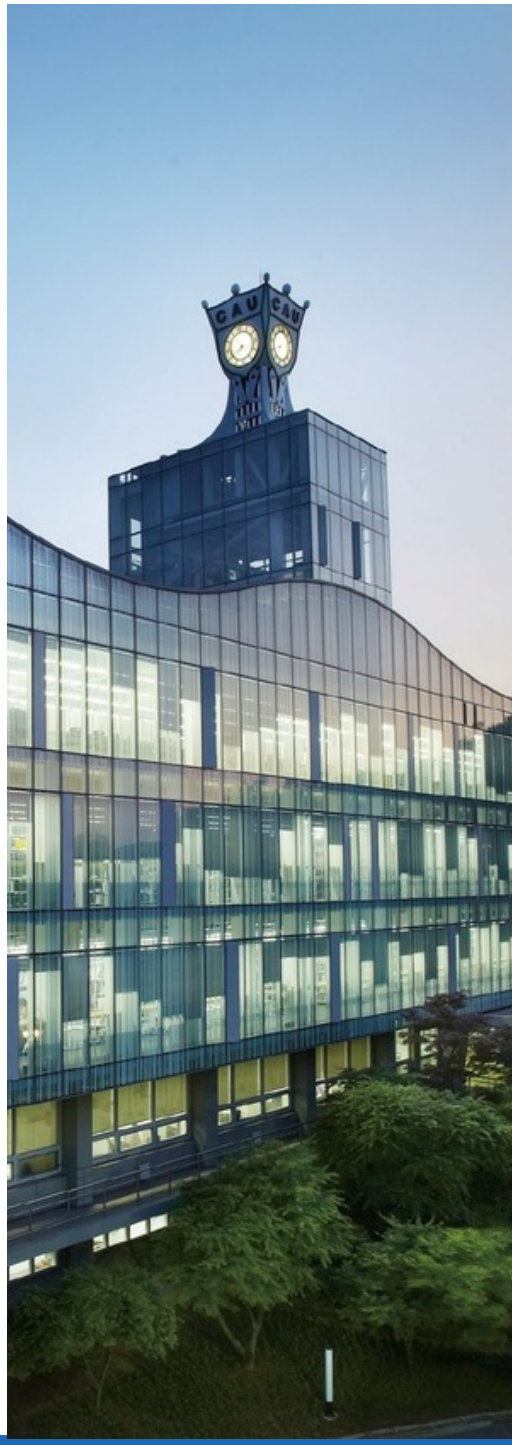
Principles of Programming Languages

Lecture 3
Describing Syntax and Semantics (구문과 의미론)

Jinsung Kim, Ph.D.
High-Performance Computing (HPC) Lab
Computer Science and Engineering
Chung-Ang University



Static Semantics (정적 의미론)

- **Nothing to do with meaning**
- **Context-free grammars (CFGs) cannot describe all of the syntax of programming languages**
- **Categories of constructs that are trouble:**
  - Context-free, but cumbersome (e.g., types of operands in expressions)
  - Non-context-free (e.g., variables must be declared before they are used)
- **Static semantics**
  - Indirectly related to the meaning of programs (프로그램의 의미와 간접적으로 관련됨)
  - The legal forms of programs (프로그램의 적법한 형식)

Attribute Grammars (속성 문법)

- **Attribute grammars (AGs) have *additions* to CFGs to carry some semantic information on parse tree nodes**

- **Primary value of AGs:**
  - Static semantics specification (정적 의미론을 명세)
  - Compiler design (static semantics checking) (컴파일 시점에 검사)

Attribute Grammars: Definition (1/2)

- **Def.: An attribute grammar is a context-free grammar $G=(S,N,T,P)$ with the following *additions*:**
  - S: the start symbol (a single nonterminal from N)
  - N: the set of nonterminal symbols (variables)
  - T: the set of terminal symbols (tokens/lexemes)
  - P: the set of productions

  - Associated with each grammar symbol X is a set of attributes $A(X)$
  - Associated with each grammar rule is a set of semantic functions (의미 함수) and a possibly empty set of predicate functions (술어함수) over the attributes of the symbols in the grammar rule.
  - A predicate function has the form of a Boolean expression on the union of the attribute set $\{A(X_0), \ldots, A(X_n)\}$ and a set of literal attribute values
  - The parse tree is fully attributed if all the attribute values are computed

Attribute Grammars: Definition (2/2)

- Let $X_0 \rightarrow X_1 \ \ldots \ X_n$ be a rule
- Functions of the form $S(X_0) = f(A(X_1), \ldots, A(X_n))$ define synthesized (합성) attributes
- Functions of the form $I(X_j) = f(A(X_0), \ldots, A(X_n))$, for $1 \leq j \leq n$, define inherited (상속) attributes
- Initially, there are intrinsic (내장) attributes on the leaves
- A predicate function has the form of a Boolean expression on the union of the attribute set $\{A(X_0), \ldots, A(X_n)\}$ and a set of literal attribute values

Attribute Grammars: An Example (1/2)

- **Syntax**

```
<assign> → <var> = <expr>
<expr>   → <var> + <var>
         | <var>
<var>    → A | B | C
```

- `actual_type`**: synthesized for** `<var>` **and** `<expr>`
- `expected_type`**: inherited for** `<expr>`

Attribute Grammars: An Example (2/2)

- **Syntax rule**

  `<expr> → <var>[1] + <var>[2]`

  - Semantic rules:

    `<expr>.actual_type ← <var>[1].actual_type`

  - Predicate:

    `<var>[1].actual_type == <var>[2].actual_type`

    `<expr>.expected_type == <expr>.actual_type`

- **Syntax rule**

  `<var> → A | B | C`

  - Semantic rule:

    `<var>.actual_type ← lookup (<var>.string)`

    - The look-up function looks up a given variable name in the symbol table and returns the variable's type

Attribute Grammars

- **How are attribute values computed?**
  - If all attributes were inherited, the tree could be decorated in top-down order
  - If all attributes were synthesized, the tree could be decorated in bottom-up order
  - In many cases, both kinds of attributes are used, and it is some combination of top-down and bottom-up that must be used

Attribute Grammars

<expr>.expected_type ← inherited from parent

<expr>.expected_type = <var>.actual_type

<var>[1].actual_type ← lookup (A)

<var>[2].actual_type ← lookup (B)

<var>[1].actual_type =? <var>[2].actual_type

<expr>.actual_type ← <var>[1].actual_type

<expr>.actual_type =? <expr>.expected_type

Semantics (동적 의미론)

- **Meaning of the expressions, statements, and program units of a programming language (execution rules)**
- **There is *no* single widely acceptable notation or formalism for describing semantics**
- **Several needs for a methodology and notation for semantics:**
  - Programmers need to know what statements mean
  - Compiler writers must know exactly what language constructs do
  - Correctness proofs would be possible (Axiomatic, 공리의미론)
  - Compiler generators would be possible
  - Designers could detect ambiguities and inconsistencies

Semantics (동적 의미론)

- **Primary Methods**
  - Three main mathematical and logical approaches are used

  - **Operational Semantics (연산 의미론)**
    - Describes the meaning of language constructs by specifying their effects on an ideal machine, focusing on the sequence of changes in the machine's state

  - **Denotational Semantics (표기 의미론)**
    - Describes meaning by mapping syntactic elements to specific mathematical objects (such as recursive functions) that represent their concrete meanings

  - **Axiomatic Semantics (공리 의미론)**
    - Based on formal logic, it describes meaning by defining axioms and inference rules– such as preconditions and postconditions for each statement– to mathematically prove a program's correctness

Operational Semantics

- **Operational Semantics (연산 의미론)**
  - Describe the meaning of a program by executing its statements on a machine, either simulated or actual.
  - The change in the state of the machine (memory, registers, etc.) defines the meaning of the statement
  - Depend on programming languages of lower levels, not mathematics
    - The statements of one programming language are described in terms of the statements of a lower-level programming language

- **To use operational semantics for a high-level language, a virtual machine is needed**

Operational Semantics

- **Example:** `for` **structure in C**

C Statement

```
for (expr1; expr2; expr3) {
   . . .
}
```

Meaning

```
        expr1;
loop: if expr2 == 0 goto out
        . . .
        expr3;
        goto loop
out:    . . .
```

Operational Semantics

- **Evaluation**
  - Good if used informally (language manuals, etc.)
  - Extremely complex if used formally (e.g.,VDL (PL/I))

Denotational Semantics (표기 의미론)

- **Based on recursive function theory**
- **The process of building a denotational specification for a language:**
  - Define a mathematical object for each language entity
  - Define a function that maps instances of the language entities onto instances of the corresponding mathematical objects (denotational)
    - Syntactic domain (구문 정의역)
    - Semantic domain (의미 정의역)
- **The meaning of language constructs are defined by only the values of the program's variables**

Denotational Semantics: Program State

- **The state of a program is the values of all its current variables**

$$s = \{\langle i_1, v_1\rangle, \langle i_2, v_2\rangle, \ldots, \langle i_n, v_n\rangle\}$$

- **Let `VARMAP` be a function that, when given a variable name and a state, returns the current value of the variable**

$$\text{VARMAP}(i_j, s) = v_j$$

Decimal Numbers

<dec_num> → '0' | '1' | '2' | '3' | '4' | '5' |
'6' | '7' | '8' | '9'
| <dec_num> ('0' | '1' | '2' | '3' | '4' | '5' |
'6' | '7' | '8' | '9')

M_{dec} ('0') = 0, M_{dec}('1') = 1, …, M_{dec}('9') = 9

M_{dec} (<dec_num> '0') = 10 * M_{dec}(<dec_num>)

M_{dec} (<dec_num> '1') = 10 * M_{dec}(<dec_num>) + 1

…

M_{dec} (<dec_num> '9') = 10 * M_{dec}(<dec_num>) + 9

Logical Pretest Loops

- **Maps state sets to state sets ∪ {"error"}**

```
M_l(while B do L, s) Δ=
          if M_b(B, s) == undef
          then error
          else if M_b(B, s) == false
               then s
               else if M_sl(L, s) == error
                    then error
                    else M_l(while B do L, M_sl(L, s))
```

Evaluation of Denotational Semantics

- Can be an aid to language design
- Because of its complexity, it is of little use to language users
- Provides an excellent way to describe a language precisely

Axiomatic Semantics (공리 의미론)

- **Based on formal logic (predicate calculus, 술어 계산학)**
  - Formal logic (형식 논리) provides the symbolic rules to define, reason about, and verify program behavior
- **Original purpose: formal program verification**
  - Program verification and program semantics specification
- **Hoare Logic**

$$\{P\}\ C\ \{Q\}$$

- **Axioms or inference rules are defined for each statement type in the language**

Axiomatic Semantics

- The logic expressions are called *assertions* (단언 또는 술어)
- An assertion before a statement (a precondition, 전조건) states the relationships and constraints among variables that are true at that point in execution
- An assertion following a statement is a postcondition (후조건)

```
Pre-, post form: {P} statement {Q}
```

- A *weakest* precondition (최약 전조건) is the least restrictive precondition that will guarantee the postcondition

Axiomatic Semantics Form

- **An example**

```
a = b + 1   {a > 1}
```

  - One possible precondition: `{b > 10}`
  - Weakest precondition: `{b > 0}`

Axiomatic Semantics Form

- **An example**

```
sum = 2 * x + 1   {sum > 1}
```

  - One possible precondition: `{x > 10}`
  - Weakest precondition: `{x > 0}`

Program Proof Process

- **The postcondition for the entire program is the desired result**
  - Work back through the program to the first statement.
  - If the precondition on the first statement is the same as the program specification, the program is correct.

- **Inference rule (추론 규칙)**
  - A method of inferring the truth of one assertion on the basis of the values of other assertions

$$\frac{S_1, S_2, \ \ldots, S_n}{S}$$

Axiomatic Semantics: Assignment

- **An axiom for assignment statements**

$$(x=E): \{Q_{x \to E}\}\ x=E\ \{Q\}$$

$$P = Q_{x \to E}$$

- Example: `a = b / 2 - 1 {a < 10}`
- Example: `x = x + y - 3 {x > 10}`

- **A Logical statement (or theorem)**
  - A given assignment state with both a pre- and post-conditions
  - If the assignment axiom, when applied to the postcondition and the assignment statement, produces the given precondition, the theorem is proved.

Axiomatic Semantics:

- **Examples**
  - `{x > 3} x = x - 3 {x > 0}`
  - `{x > 5} x = x - 3 {x > 0} ???`

Axiomatic Semantics:

- **The rule of consequence (결과 규칙):**

$$\frac{\{P\}\ S\ \{Q\},\ P' \Rightarrow P,\ Q \Rightarrow Q'}{\{P'\}\ S\ \{Q'\}}$$

  - Example: $\{x > 5\}\ x = x - 3\ \{x > 0\}$

Axiomatic Semantics: Sequences

- **An inference rule for sequences of the form (시퀀스 구조)**

```
S1;S2
```

```
{P1} S1 {P2}
{P2} S2 {P3}
```

$$\frac{\{P1\}\ S1\ \{P2\},\quad \{P2\}\ S2\ \{P3\}}{\{P1\}\ S1;S2\ \{P3\}}$$

Axiomatic Semantics: Sequences

- **An inference rule for sequences of the form**

```
S1;S2
```

```
{P1} S1 {P2}
{P2} S2 {P3}
```

$$\frac{\{P1\}\ S1\ \{P2\},\quad \{P2\}\ S2\ \{P3\}}{\{P1\}\ S1;S2\ \{P3\}}$$

- Example: `S1: x1 = E1, S2: x2 = E2`
  - `y = x * 3 +1; x = y + 3; {x < 10}`

Axiomatic Semantics: Selection

- **An inference rules for selection**

```
if B then S1 else S2
```

$$\frac{}{\{P\}\ \texttt{if B then S1 else S2}\ \{Q\}}$$

Axiomatic Semantics: Selection

- **An inference rules for selection**

```
if B then S1 else S2
```

$$\frac{\{B \textbf{ and } P\}\ S1\ \{Q\},\quad \{(\textbf{not } B) \textbf{ and } P\}\ S2\ \{Q\}}{\{P\}\ \textbf{if } B \textbf{ then } S1 \textbf{ else } S2\ \{Q\}}$$

- Example: `if x > 0 then y = y - 1 else y = y + 1 {y > 0}`

Axiomatic Semantics: Logical Pretest Loops

- **An inference rule for logical pretest loops (논리 사전-검사 루프)**

```
{P} while B do S end {Q}
```

$$\frac{\texttt{(I and B) S \{I\}}}{\texttt{\{I\} while B do S \{I and (not B)\}}}$$

, where `I` is the **loop invariant(루프 불변자)** (the inductive hypothesis)

Axiomatic Semantics:

- **Characteristics of the loop invariant: `I` must meet the following conditions:**
  - `P ⇒ I` : the loop invariant must be true initially
  - `{I and B} S {I}` : `I` is not changed by executing the body of the loop
  - `(I and (not B)) ⇒ Q` : if `I` is true and `B` is false, `Q` is implied
  - The loop terminates : can be difficult to prove

```
wp(stmt, post-cond.) = pre-cond
```

Axiomatic Semantics:

- **Example**

```
while y <> x do y = y + 1 end {y = x}
```

Loop Invariant

- **The loop invariant `I` is a weakened version of the loop postcondition, and it is also a precondition**
- **`I` must be weak enough to be satisfied prior to the beginning of the loop, but when combined with the loop exit condition, it must be strong enough to force the truth of the postcondition**

- **Total/Partial correctness (전체/부분 정확성)**
  - With or without proving loop termination

Evaluation of Axiomatic Semantics

- Developing axioms or inference rules for all of the statements in a language is difficult
- It is a good tool for correctness proofs, and an excellent framework for reasoning about programs, but it is not as useful for language users and compiler writers

Denotation Semantics vs. Operational Semantics

- In operational semantics, the state changes are defined by coded algorithms
- In denotational semantics, the state changes are defined by rigorous mathematical functions

Summary

- **BNF and context-free grammars are equivalent meta-languages**
  - Well-suited for describing the syntax of programming languages
- **An attribute grammar is a descriptive formalism that can describe both the syntax and the semantics of a language**
- **Three primary methods of semantics description**
  - Operation, axiomatic, denotational

Q & A